

Power Analyses using Monte Carlo Simulations

An overview and brief tutorial

R. Philip Chalmers

Introduction

Why statistical power matters:

- **Study design** and **resource allocation** (determine required sample size to obtain sufficient power/precision in model estimates)
- **Study sensitivity** (how big an effect needs to be before it's reliably detected)
- **Interpreting null results** (high-powered studies that still fail to reject H_0 are informative)
- Resource optimization, such via **cost-benefit analyses**
- Comparing between **competing designs** (within vs between ANOVA; robust vs liberal estimators; etc)

These and other reasons are why power estimates are frequently *required for preregistration*.

Statistical Power

Under the H_0 perspective, power is understood as

$$P(\text{reject } H_0 | H_1 \text{ is true})$$

when “rejecting the null” using $f(D_\psi | \psi_0) < \alpha$ is the target analysis. More precisely, power (the complement of a Type II error, $1 - \beta$) is

$$1 - \beta = \mathcal{E}(f(D_\psi | \psi_0) < \alpha)$$

- Ultimately, power = probability of “*data surprise*” implied by H_1 ($f(D_\psi | \psi_0) \rightarrow$ *p-value*); “surprise” controlled by α tolerance
 - Higher-power = greater chance of being surprised by H_0

Types of power analyses

- **Retrospective (post-hoc) and prospective power**
 - Evaluate how powerful prior published work was (Cohen, 1968)
 - Relatedly, there's the topic of *observed power* ... but please don't do this
- **A priori analyses** (e.g, sample size resource planning)
- **Sensitivity analyses** (how large effect size must be before it's reliably detected/quantified)

Types of power analyses

- **Retrospective (post-hoc) and prospective power**
 - Evaluate how powerful prior published work was (Cohen, 1968)
 - Relatedly, there's the topic of *observed power* ... but please don't do this
- **A priori analyses** (e.g., sample size resource planning)
- **Sensitivity analyses** (how large effect size must be before it's reliably detected/quantified)

If modifying α is justifiable:

- **Compromise analyses** (trade-offs between α and β errors)
- **Criterion analyses** (adjust α to meet target power)

Common software

Power analysis sometimes have analytic (or easily expressible) non-centrality distributions that can be used to solve the inputs and outputs in the target experiment.

- Common software capitalizes on this feature directly: `pwr` in R, G*Power 3.1, PASS, SAS POWER, Stata, etc
- Given some target, and assuming there is only one missing variable in the power equation, *solve using known non-centrality distributions* (primarily via root-finding algorithms)
 - Assumes **canonical (exponential) distributions** for random variables, and that **specific analyses** are used for inference in each dataset

Common software

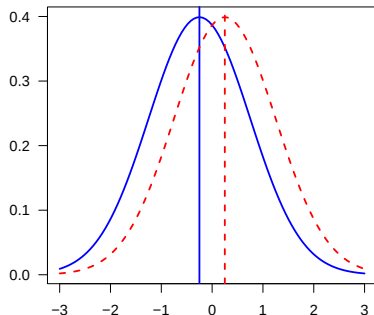
Here's an example using the `pwr` package in R, performing an independent samples t -test with $n = 60$ observations per group, and with a Cohen's d of 0.5.

```
pwr::pwr.t.test(n=60, d=.5)
```

```
##  
##      Two-sample t test power calculation  
##  
##              n = 60  
##              d = 0.5  
##      sig.level = 0.05  
##              power = 0.7752659  
##      alternative = two.sided  
##  
## NOTE: n is number in *each* group
```

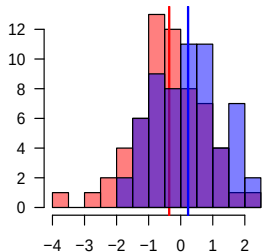
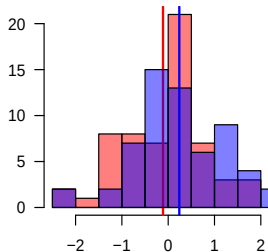
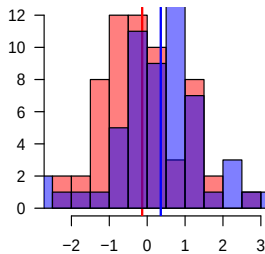
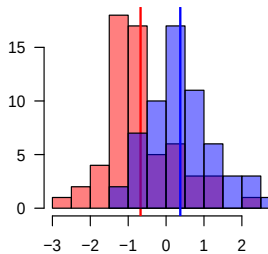
Population distributions

Standardized mean difference of $d = 0.5$



Equal sample sizes and variances assumed

Samples from population distributions



Common software

Same setup, however in this case obtaining the associated sample size that would give a power of $1 - \beta = .95$ (a priori power analysis)

```
pwr::pwr.t.test(n=NULL, d=.5, power=.95)
```

```
##  
##      Two-sample t test power calculation  
##  
##              n = 104.9279  
##              d = 0.5  
##      sig.level = 0.05  
##              power = 0.95  
##      alternative = two.sided  
##  
## NOTE: n is number in *each* group
```

Power via simulation

Why Power via simulation?

Classical approaches to power are attractive due to their convenience and compute speed, but they are strongly restricted to a constrained class of (mostly linear) models.

Power via simulation

Why Power via simulation?

Classical approaches to power are attractive due to their convenience and compute speed, but they are strongly restricted to a constrained class of (mostly linear) models.

- 1) **Necessary:** Many modeling applications too complex, resulting in difficult non-central distributions (intractable)

Power via simulation

Why Power via simulation?

Classical approaches to power are attractive due to their convenience and compute speed, but they are strongly restricted to a constrained class of (mostly linear) models.

- ① **Necessary:** Many modeling applications too complex, resulting in difficult non-central distributions (intractable)
- ② **Realistic:** Estimate power across diverse research contexts
 - Can even embed the *behaviour* of analysts (e.g., omnibus ANOVA first → post-hoc test)

Power via simulation

Why Power via simulation?

Classical approaches to power are attractive due to their convenience and compute speed, but they are strongly restricted to a constrained class of (mostly linear) models.

- ① **Necessary:** Many modeling applications too complex, resulting in difficult non-central distributions (intractable)
- ② **Realistic:** Estimate power across diverse research contexts
 - Can even embed the *behaviour* of analysts (e.g., omnibus ANOVA first → post-hoc test)
- ③ **Intuitive:** Despite the initial setup, power by simulation is very easy to understand

Power via simulation

Why Power via simulation?

Classical approaches to power are attractive due to their convenience and compute speed, but they are strongly restricted to a constrained class of (mostly linear) models.

- ① **Necessary:** Many modeling applications too complex, resulting in difficult non-central distributions (intractable)
- ② **Realistic:** Estimate power across diverse research contexts
 - Can even embed the *behaviour* of analysts (e.g., omnibus ANOVA first → post-hoc test)
- ③ **Intuitive:** Despite the initial setup, power by simulation is very easy to understand
- ④ Becoming more **available** all the time (`simr`, `semPower`, `mlmpower`, `power4mome`, `powerMediation`, `Superpower`)

Power via simulation

If the goal is to simply *estimate* power, $1 - \hat{\beta}$ can be obtained via:

$$1 - \hat{\beta} = \frac{\sum_{r=1}^R f_r(D_\psi | \psi_0) < \alpha}{R}$$

- In words, this is the **proportion of times the p -values across R independently generated datasets are less than α**

Power via simulation

If the goal is to simply *estimate* power, $1 - \hat{\beta}$ can be obtained via:

$$1 - \hat{\beta} = \frac{\sum_{r=1}^R f_r(D_\psi | \psi_0) < \alpha}{R}$$

- In words, this is the **proportion of times the p -values across R independently generated datasets are less than α**

$1 - \hat{\beta}$ is just an estimate of the true $1 - \beta$, so it's important that:

- a) R is large enough (e.g., 5000+), and
- b) the expected sampling variability is reported

$$\widehat{SE}_{(1-\hat{\beta})} = \sqrt{\frac{\hat{\beta}(1 - \hat{\beta})}{R}}$$

Power via simulation

Steps for the simulation approach:

- ① Define **data-generation process** reflecting the *expected* empirical scenario under H_1 to obtain D_ψ samples
 - Most often these are informed by your analysis/model
 - If sampling distributions not well understood, falling back to canonical distribution assumptions *may* reasonable . . .
- ② Define **analysis plan(s)** to use on the D_ψ sample (e.g., SEM, IRT, mixed effect model, etc)
- ③ Extract inference criterion (p -values, posterior probabilities, Bayes factors, logical state evaluations [e.g., SE/CI precision satisfied?])

Repeat above steps R independent times, studying the average behavior across the random samples.

Power via simulation

Here's a very simple example using an independent samples t -test design (completely unnecessary, but presented for general structure).

Independent samples t -test via simulation

Function generates and analyses data given specific inputs, returning a p -value (n = sample size per group; d is Cohen's d).

```
p_ind.t.test <- function(n, d){  
  
  # Generate data  
  g1 <- rnorm(n)  
  g2 <- rnorm(n, mean=d)  
  
  # Analyse generated data  
  p <- t.test(g2, g1, var.equal=TRUE)$p.value  
  p  
}
```

Power via simulation

```
# p-values across different generated datasets
```

```
p_ind.t.test(60, .5)
```

```
## [1] 0.4856808
```

```
p_ind.t.test(60, .5)
```

```
## [1] 0.05656728
```

```
p_ind.t.test(60, .5)
```

```
## [1] 0.03923165
```

Again, how often these p -values are less than the desired α cutoff = estimate of the power.

Power via simulation

WARNING: HEAVY CODE LIFTING AHEAD!

simulate R = 10,000 times to get estimate of (1 - beta)

```
R <- 10000
```

```
ps <- replicate(R, p_ind.t.test(n=60, d=.5))
```

```
alpha <- .05
```

```
power_hat <- mean(ps < alpha)
```

```
SE_hat <- sqrt(power_hat * (1-power_hat) / R)
```

```
data.frame(n=60, d=.5, alpha, power_hat, SE_hat,  
           CI_2.5=power_hat + qnorm(.025)*SE_hat,  
           CI_97.5=power_hat + qnorm(.975)*SE_hat)
```

```
##      n    d alpha power_hat      SE_hat    CI_2.5    CI_97.5  
## 1 60 0.5  0.05      0.784 0.004115143 0.7759345 0.7920655
```

True value from earlier: $1 - \beta = 0.7752$.

Other power analyses via simulation

A prior power analysis, on the other hand, requires solving

$$\frac{\sum_{r=1}^R f_r(D_\psi | \psi_0) < \alpha}{R} = \text{target power}$$

by picking the right n to generate D_ψ .

Other power analyses via simulation

A prior power analysis, on the other hand, requires solving

$$\frac{\sum_{r=1}^R f_r(D_\psi | \psi_0) < \alpha}{R} = \text{target power}$$

by picking the right n to generate D_ψ .

- **More problematic!** Requires multiple iterations across competing n s
 - Time/resource consuming if not done efficiently (e.g., via stochastic root solving; see Chalmers, 2024), and often **prone to solutions that are far from the target!**
- Sensitivity power analysis has same issue, varying ψ instead of n

Power via simulation

Why NOT Power via simulation?

- 1) **More responsibility** on the researcher's side to provide specifics
 - IMO this is double-edged, as blind application using automated software can be worse . . .

Power via simulation

Why NOT Power via simulation?

- ① **More responsibility** on the researcher's side to provide specifics
 - IMO this is double-edged, as blind application using automated software can be worse . . .
- ② More **time consuming** to obtain accurate estimates

Power via simulation

Why NOT Power via simulation?

- ① **More responsibility** on the researcher's side to provide specifics
 - IMO this is double-edged, as blind application using automated software can be worse . . .
- ② More **time consuming** to obtain accurate estimates
- ③ Often more **savvy programming skillz** required (particularly for custom applications)

Power via simulation

Why NOT Power via simulation?

- ① **More responsibility** on the researcher's side to provide specifics
 - IMO this is double-edged, as blind application using automated software can be worse . . .
- ② More **time consuming** to obtain accurate estimates
- ③ Often more **savvy programming skillz** required (particularly for custom applications)
- ④ **Fewer established standards** (e.g., size of R , target SEs/CIs , stochastic root-solving approaches)

Power via simulation

Why NOT Power via simulation?

- ① **More responsibility** on the researcher's side to provide specifics
 - IMO this is double-edged, as blind application using automated software can be worse ...
- ② More **time consuming** to obtain accurate estimates
- ③ Often more **savvy programming skillz** required (particularly for custom applications)
- ④ **Fewer established standards** (e.g., size of R , target SEs/CIs , stochastic root-solving approaches)
- ⑤ Of the simulation software available, they **vary in quality**
 - Estimate $1 - \beta$ only; no natural means for a priori analyses
 - Missing power curves, parallel processing, batch modes, ...

Spower

To address the implementation complications that come with simulation-based power, I've created the **Spower** package in R, now available on CRAN.

- **A general purpose simulation-based power analysis API** for routine and customized simulation designs
- Focuses exclusively on simulation approaches to (expected) prospective power analyses, criterion analyses, compromise analyses, sensitivity analyses, and a priori/post-hoc analyses

Spower package

- **Simple API:** three main functions (`Spower()`, `SpowerBatch()`, and `SpowerCurve()`), along with other helper functions
- **Full customization:** can be applied to any experimental setup (more advanced, but most flexible approach)
- **Built-in common power analyses** (e.g., those found in G*Power 3.1)
- Package is generally designed for ease of application, efficiency, accuracy, and features (e.g., parallel processing, re-running analyses, customization, etc).

All that is required is a suitable function that returns p -values, posterior probabilities, or logicals (TRUE/FALSE)

Demonstrations

Obtaining an estimate of power just requires passing the `p_*`() function to the titular function `Spower()`, which performs the experiment $R = 10,000$ times by default. Function can be piped as well, which generally makes the evaluation more readable.

```
# install.packages('Spower')  
library(Spower)  
  
# wrapped  
Spower(p_ind.t.test(n=60, d=.5))  
  
# passed via the |> pipe  
p_ind.t.test(n=60, d=.5) |> Spower()
```

Demonstrations

```
p_ind.t.test(n=60, d=.5) |> Spower()
##
## -- Spower Results -----
##
## Design conditions:
##
## # A tibble: 1 x 4
##       n      d sig.level power
##   <dbl> <dbl>   <dbl> <lgl>
## 1    60  0.5     0.05 NA
##
## Estimate of power: 0.779
## 95% Confidence Interval: [0.771, 0.787]
## Execution time (H:M:S): 00:00:04
```


A priori analysis

When specific power criteria are to be solved, additional arguments are passed to `Spower()`. In this case, estimating the sample size that is required to obtain a power of $1 - \beta = .95$. However, limit the search¹ to fall within $n = [10, 200]$

```
# use interval() to search for value  
p_ind.t.test(n = interval(10, 200), d = .5) |>  
  Spower(power = .95)
```

```
# this is equivalent (matches the output)  
# p_ind.t.test(n = NA, d = .5) |>  
#   Spower(power = .95, interval=c(10, 200))
```

¹Interval can be very large; the stochastic root-solver is quite forgiving!

A priori analysis

```
p_ind.t.test(n = interval(10, 200), d = .5) |>
  Spower(power = .95)
##
## -- Spower Results -----
##
## Design conditions:
##
## # A tibble: 1 x 4
##       n      d sig.level power
##   <dbl> <dbl>   <dbl> <dbl>
## 1    NA  0.5     0.05  0.95
##
## Estimate of n: 104.8
## 95% Confidence Interval: [102.3, 107.6]
## Execution time (H:M:S): 00:00:12
```

Sensitivity, Criterion, and Compromise Power

```
# sensitivity analysis (solve effect size d)  
p_ind.t.test(n=60, d = interval(.01, 3)) |>  
  Spower(power = .95)  
  
# criterion (interval not required as alpha bounded [0,1])  
p_ind.t.test(n=60, d = .5) |>  
  Spower(power = .95, sig.level = NA)  
  
# compromise analysis (beta/alpha = q, trade-off ratio)  
p_ind.t.test(n=60, d = .5) |>  
  Spower(beta_alpha = 2)
```

Customized vs built-in

Fortunately, re-inventing the simulation wheel is not always necessary.

- **Spower** currently supports all subroutines found in the popular G*Power 3.1 software, where all examples in the manual are currently presented as a [vignette on CRAN](#)
 - Data generation of these functions can be modified via **drop-in data generation** (replacing `gen_*` arguments)

Customized vs built-in

Fortunately, re-inventing the simulation wheel is not always necessary.

- **Spower** currently supports all subroutines found in the popular G*Power 3.1 software, where all examples in the manual are currently presented as a [vignette on CRAN](#)
 - Data generation of these functions can be modified via **drop-in data generation** (replacing `gen_*` arguments)
- Other power analysis approaches also available:
 - ROPEs, equivalence testing, Type S/M errors and their power compliments (Gelman, 2013)
 - **Expected power** also naturally available to include prior uncertainty in target effects sizes or auxiliary model parameters

Expected power

Suppose that you didn't actually know the true d parameter in the population, but from past research it tended to vary between .4 and .6. Rather than assuming d , generate the uncertainty in the computations to find the average/expected power.

```
# assume  $d \sim N(.5, .1)$ 
p_ind.t.test(n=60, d = rnorm(1, mean=.5, sd=.1)) |> Spower()

# can be further functionalized!
d_prior <- function() rnorm(1, mean=.5, sd=.1)
p_ind.t.test(n=60, d=d_prior()) |> Spower()
```

Expected power

Evaluation leads to a lower power than with the fixed d input as the uncertainty in the effect size is averaged over. Similar idea can be used for all aspects of the model (e.g., uncertainty in the variances).

```
p_ind.t.test(n=60, d=d_prior()) |> Spower()
```

```
##  
## -- Spower Results -----  
##  
## Design conditions:  
##  
## # A tibble: 1 x 3  
##       n sig.level power  
##   <dbl>   <dbl> <lgl>  
## 1     60     0.05 NA  
##  
## Estimate of power: 0.737  
## 95% Confidence Interval: [0.728, 0.745]  
## Execution time (H:M:S): 00:00:04
```

Expected power

A priori analysis using expected power instead (slightly higher n).

```
p_ind.t.test(n=interval(5, 200), d=d_prior()) |> Spower(power=.8)
##
## -- Spower Results -----
##
## Design conditions:
##
## # A tibble: 1 x 3
##       n sig.level power
##   <dbl>   <dbl> <dbl>
## 1    NA     0.05  0.8
##
## Estimate of n: 71.1
## 95% Confidence Interval: [70.3, 71.8]
## Execution time (H:M:S): 00:00:18
```


SpowerBatch() and SpowerCurve()

Running `Spower()` in batch mode to evaluate multiple inputs to create **sets of analyses** and **graphical outputs**.

```
# evaluate across three different sample sizes  
p_ind.t.test(d = .5) |> SpowerBatch(n=c(25, 50, 75))
```

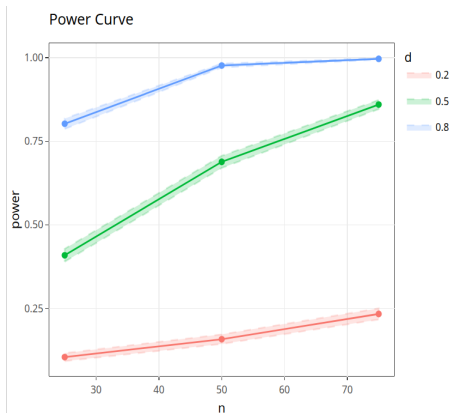
```
# evaluate across three different sample/effect sizes  
p_ind.t.test() |> SpowerBatch(n=c(25, 50, 75), d=c(.2, .5, .8))
```

Similarly, jump straight to plotting!

```
p_ind.t.test() |> SpowerCurve(n=c(25, 50, 75), d=c(.2, .5, .8))
```

SpowerCurve()

Graphic produced is interactive, so hovering over the element of interest will provide estimates and their uncertainty.



Closing

If you're using R analyses functions already, then half the work is basically done before using **Spower**.

- All you need is to write a function with a suitable data-generation step and extract out the p -values!
- LLMs like ChatGPT or Claude can also help you code these simple p_* functions!
- **Power evaluations completely automated, ultimately removing clunky/error prone coding issues**

For more information see:

- *Chalmers, R.P. (2025). Spower: A general-purpose Monte Carlo simulation power analysis program. Behavior Research Methods, 57, 348. DOI: 10.3758/s13428-025-02787-z*